

Sentinel Flow: AI + N8N Workflows for Security Operations

P-2024-28-CS-123| Frozen canonical title: AI + N8N Workflows for Security Operations

LAKSHAN KUMARESH V D

School of Innovation, KGiSL Institute of
Technology, Coimbatore, India

RITHISH S

School of Innovation, KGiSL Institute of
Technology, Coimbatore, India

LOGA MUMMOORTHY S

Kanishk Upreti (SoI Project Mentor)
School of Innovation, KGiSL Institute of
Technology, Coimbatore, India

Abstract—This paper presents an automated cybersecurity incident response workflow built on the self-hosted n8n platform. The workflow ingests security alerts from SIEM, EDR and firewall sources, normalizes them to a common taxonomy, enriches indicators with VirusTotal, AbuseIPDB and Shodan, computes a composite threat score (0–100), and uses a large language model (LLM) to return a structured severity assessment and playbook recommendation. High-severity incidents branch to containment actions (IP block, host isolation, session revocation), which are simulated with mock endpoints. Slack notification, Jira ticket creation and a Google Sheets audit log complete the pipeline. End-to-end operation was verified in a controlled local environment using test alerts. The results demonstrate feasibility only and do not establish performance against real attacks, production integrations or large alert volumes. Future work centers on a human approval gate, credential management, real endpoint integration and quantitative evaluation.

Keywords—security orchestration, incident response automation, n8n, threat intelligence enrichment, LLM severity assessment, SOAR, audit logging

I. INTRODUCTION AND PROBLEM CONTEXT

Security operations centers (SOCs) face high alert volumes and slow manual triage. For each alert, analysts must correlate indicators with threat intelligence, judge severity, notify colleagues, open a ticket and run containment steps by hand. This delays response, increases analyst fatigue and produces inconsistent records that are hard to audit [1], [4].

This project addresses that gap with a workflow that automates the repeatable parts of triage and response while keeping each decision traceable. The system is implemented in n8n, a workflow automation platform, and connects commodity security services through HTTP APIs. The objectives were to:

- (1) ingest security alerts from SIEM, EDR and firewall sources;
- (2) enrich indicators with threat intelligence;
- (3) use an LLM to assess severity and recommend a response playbook;
- (4) automatically execute containment for high-severity incidents;
- (5) notify the SOC and create audit-ready tickets and logs.

Contributions. (i) A four-stage, end-to-end incident response pipeline assembled from open tooling and free-tier services; (ii) a composite threat score that fuses three intelligence sources and tolerates partial API failure; (iii) a structured-output LLM assessment stage with guarded parsing; and (iv) a documented set of implementation issues and mitigations that may help others building similar low-code SOAR pipelines.

The MVP is a standalone, locally hosted workflow. Containment endpoints are mocks, and evaluation was functional and limited in scope.

The problem is not only speed. Manual response depends on the individual analyst on shift, so the same alert can be rated differently, documented differently or escalated late. An automated pipeline can apply the same enrichment steps, the same taxonomy and the same record format to every alert. It can also create a consistent evidence trail for later review, which is a core expectation of incident handling guidance [1]. At the same time, automation that acts without oversight can cause harm, for example by blocking a legitimate address. The project therefore treats safe automation, and not only fast automation, as a design goal.

The work is scoped as a controlled proof of concept. It was built by a student team on a local n8n instance, with free-tier or trial access to external services. Hardware appliances, production firewalls, enterprise identity providers, dashboards and large-scale deployment were outside the MVP scope. The rest of the paper is organized as follows. Section II positions the work, Section III traces its six development stages, Section IV describes the architecture, Section V the methodology, Sections VI and VII the implementation status and evaluation protocol, and Sections VIII–XI the trade-offs, risks, limitations and conclusion.

II. RELATED WORK AND DESIGN POSITIONING

The NIST incident handling guide defines the lifecycle of preparation, detection and analysis, containment, eradication and recovery, and post-incident activity [1]. Security orchestration, automation and response (SOAR) platforms implement parts of this lifecycle through playbooks that chain enrichment, decision and response actions. Studies of SOC operations identify alert overload, tool fragmentation and manual repetition as persistent challenges [2]. The MITRE ATT&CK framework provides a common vocabulary of adversary techniques that assessments can reference [3]. Recent general-purpose LLMs, including the Gemini family [4], can follow structured prompts and return machine-readable output, which makes them candidates for alert triage and summarization. This project is positioned as a low-cost, open-tooling proof of concept that combines enrichment, LLM-assisted judgment and automated response in one auditable pipeline. It does not claim to replace commercial SOAR products, production-grade detection or analyst judgment.

Three threat intelligence services were chosen because they provide complementary views of an indicator: file and URL reputation aggregated from many engines, community-reported abuse of an address, and the exposed services and known vulnerabilities of a host [6]–[8]. A single source can be silent or wrong for a given indicator, so combining them into one score is intended to make the downstream judgment less dependent on any one feed. The LLM stage is placed after enrichment so

that the model reasons over evidence rather than over the raw alert alone.

The project therefore combines the enrichment, structured LLM assessment, conditional containment and audit logging stages in one traceable workflow. Its stated position is a controlled proof of concept for automated incident response with AI-assisted severity assessment. The project does not claim operational, commercial or hardware-validated performance. The identified limitations provide the basis for future work on a human approval gate, real endpoint integration, larger labeled alert sets and reproducible evaluation.

III. SIX-STAGE EVIDENCE EVOLUTION

Development progressed through six stages: problem identification, ideation, proof of concept, prototype, minimum viable product (MVP) and final development. Table I maps each stage to its content and to how the evidence may be used in this manuscript.

TABLE I
EVIDENCE EVOLUTION AND PERMITTED USE

Stage	Project content	Permitted use
Problem	Manual alert triage is slow and yields inconsistent, hard-to-audit records	Current team report
Ideation	Manual runbooks, commercial SOAR and a low-code build compared; n8n chosen (self-hosted, free-tier, HTTP integration)	Historical context only
Proof of concept	Webhook ingestion and alert normalization	Current team report
Prototype	Enrichment, composite scoring, first AI assessment	Current team report
MVP	Full pipeline: containment mocks, Slack, Jira, Sheets, final report	Controlled-environment report
Final	Model selection, Slack channel IDs, key rotation, approval gate	Prospective work

During implementation the language model was changed from Claude to Gemini. The AI Agent node retains the original name (Assess Severity with Claude AI), while the wired model sub-node is Google Gemini Chat Model. All reported LLM behavior therefore refers to Gemini.

A. Problem Identification

The starting concern was the delay and inconsistency of manual incident response. Analysts triaging alerts by hand must look up indicators in several tools, decide severity, contact the team and record what was done. The team framed the problem as a missing link between alert detection and coordinated response, and set the goal of an end-to-end automated pipeline that also leaves an audit-ready record. The motivating scenario is a SOC analyst handling repeated alerts, such as brute-force logins, where each alert requires the same lookups, notifications and containment steps by hand.

B. Ideation

During ideation the team considered how to connect alert sources, threat intelligence, decision making and response actions. A visual workflow platform was preferred because it allows each stage to be inspected, pinned and tested separately, and because it connects to web services through HTTP nodes without a custom integration layer. The alternatives compared were manual runbooks, commercial SOAR platforms and a custom low-code build. The low-code build on n8n was selected because it is self-hosted, uses free-tier APIs and

connects to any service over HTTP. The must-have components were alert ingestion, enrichment, AI assessment, containment triggers, notification, ticketing and logging.

C. Proof of Concept

The proof-of-concept stage established the front of the pipeline. A webhook accepted a JSON alert, and a code node validated the required fields, mapped the alert type to the standard taxonomy and generated an incident ID. This stage confirmed that alerts from different sources could be handled through one normalized structure.

D. Prototype

The prototype added the three enrichment calls, the merge step and the composite threat score, followed by a first AI assessment. It exposed the main reliability problems, such as DNS failures and crashes when an enrichment source returned an error object. These findings led to the continue-on-fail setting and the defensive coding practices described in Section V.

E. Minimum Viable Product

The MVP completed the pipeline. It added the auto-containment branch with mock firewall, EDR and session-revocation endpoints, the Slack notification, the Jira ticket, the final report, the Google Sheets audit log and the response to the caller. The MVP was verified end to end in a controlled local environment (Section VI).

F. Final Development Position

The final stage consolidates the earlier work under the title of this paper. The system should be positioned as a controlled proof of concept and not as a production SOAR deployment. The remaining actions are the model switch to gemini-1.5-flash, the Slack channel ID fix, credential rotation and the addition of a human approval gate before containment.

IV. REQUIREMENTS AND SYSTEM ARCHITECTURE

A. Functional Requirements

The system shall (R1) accept alerts by HTTP POST and reject malformed input; (R2) map alert types to a standard taxonomy and assign a unique incident ID; (R3) continue operating when an enrichment source fails; (R4) return severity, playbook and MITRE technique mapping in structured form; (R5) execute containment only when the auto-containment flag is set; and (R6) record every incident in a ticket, a notification and an audit log.

B. Pipeline Overview

The workflow has four logical stages (Fig. 1) on the n8n canvas. Each stage passes a single JSON object downstream, which keeps the data path traceable.

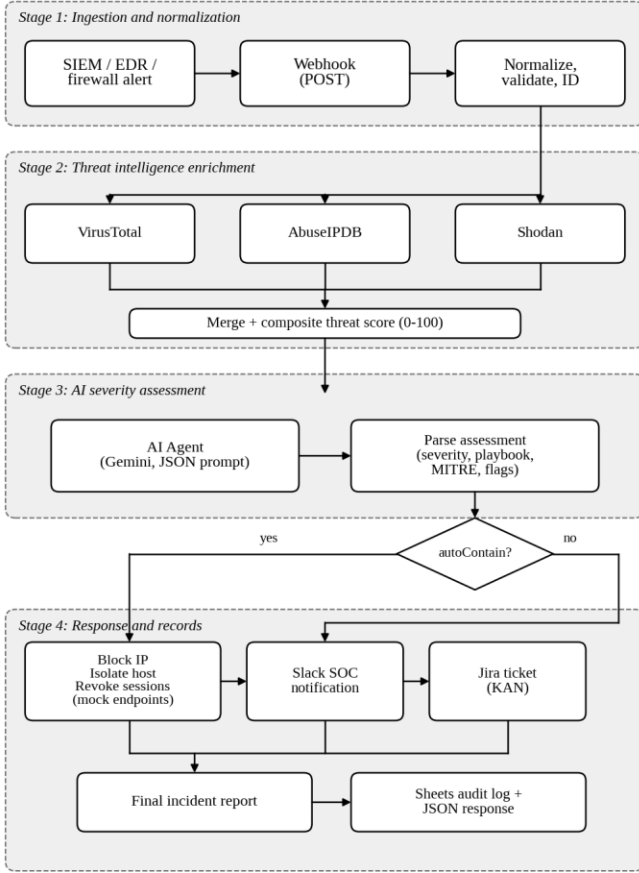


Fig. 1. Architecture of the automated incident response workflow.

C. Stage 1: Ingestion and Normalization

A webhook node receives POST alerts. A code node validates the required fields (source, alertType), maps the alert type to a standard taxonomy (e.g., BRUTE_FORCE, MALWARE DETECTED, PHISHING) and generates a unique incident ID. Invalid alerts are rejected before any external call is made.

D. Stage 2: Threat Intelligence Enrichment

Three HTTP nodes query the source IP or host (Table II). A merge node in append mode with three inputs acts as a synchronization barrier, and a code node parses the responses and computes a composite threat score from 0 to 100. Enrichment nodes are set to continue on failure, and every upstream reference is wrapped in try/catch with an empty-object default, so one API outage lowers score confidence instead of halting the response.

TABLE II
ENRICHMENT SOURCES AND ROLES

Source	Auth method	Signal used
VirusTotal	n8n predefined credential	IP reputation
AbuseIPDB	Header auth (API key)	Abuse confidence score
Shodan	API key, query parameter	Open ports, vulnerabilities, host metadata

The composite score is a weighted combination of the three normalized signals, clipped to [0, 100]: $S = \min(100, \max(0, w1 \cdot V + w2 \cdot A + w3 \cdot H))$, where V, A and H are the normalized

VirusTotal, AbuseIPDB and Shodan signals. VirusTotal contributes up to 5 points from malicious-engine detections, AbuseIPDB contributes its abuse confidence score scaled by 5, and Shodan adds 5 points for open high-risk ports or known vulnerabilities; the total is capped at 100.

E. Stage 3: AI Severity Assessment

An AI Agent node sends the normalized alert and enrichment data to Gemini with a structured prompt that requests JSON output. Because LLMs may wrap JSON in markdown fences, a parsing node strips the formatting, parses the JSON and extracts the severity, recommended playbook and MITRE ATT&CK techniques. It then sets the auto-containment and notification flags. The output schema is given in Appendix C.

F. Stage 4: Containment, Notification and Ticketing

An IF node branches on the auto-containment flag. On the containment branch, three HTTP requests block the malicious IP, isolate the affected host through an EDR interface and revoke user tokens and sessions. In the MVP these requests are sent to webhook.site mock endpoints, so payload structure can be validated without touching production systems. All branches then post a Slack message through chat.postMessage, create a Jira Cloud ticket through REST API v3, assemble a final JSON report, append an audit row to Google Sheets and return the report to the caller.

TABLE III
ARCHITECTURE AND VERIFICATION OBLIGATIONS

Layer	Responsibility	Required evidence	Failure handling
Input	Webhook, field validation	Sample payloads; test records	Reject invalid input
Enrichment	VT, AbuseIPDB, Shodan; scoring	API responses; scoring logic	Continue on fail; default to empty object
Assessment	LLM JSON output; parsing	Prompt; raw and parsed outputs	Guarded parsing; manual review
Response	Containment, Slack, Jira	Mock hits; ticket IDs	Guards on missing upstream data
Audit	Sheets log; final report	Sheet rows; JSON reports	Log even on partial failure

G. Implemented Scope Versus Planned Components

The functional boundary of the system is the workflow implemented in the MVP: webhook input, normalization, three-source enrichment, a composite score, an LLM assessment, a conditional containment branch and the notification and logging steps. Some components appear in this paper as recommendations and should not be read as implemented features. In particular, the human approval gate, real firewall, EDR and identity-provider endpoints, webhook authentication and secret management are future work. Containment in the current system is a simulated request to a mock endpoint, and no production system is modified.

V. PROJECT-SPECIFIC METHODOLOGY

A. Development Approach

Development was iterative and stage-by-stage. Each stage was tested with pinned upstream data, which isolates a failing node without re-running earlier stages. Defensive coding was added as faults surfaced: try/catch around cross-node references, explicit logical guards in place of optional chaining, and empty-object defaults for failed upstream calls. Containment was validated against mock endpoints before any production system would be touched. Credentials were

configured per service in the n8n credential store (Table IV); keys are never embedded in workflow logic.

TABLE IV
SERVICE INTEGRATIONS

Service	Integration method
Slack	Predefined Slack API credential (bot token)
Jira	Jira Software Cloud API (email, API token, domain)
Google Sheets	Service account; sheet shared with its client email
Google Gemini	API key from Google AI Studio

B. Defensive Coding and Issue Resolution

Ten implementation issues were logged. Table V summarizes the most instructive ones; the full list is given in Appendix A. Several issues shared one root cause: a downstream node assumed that an upstream node had succeeded. The corrective pattern was therefore applied everywhere, not only where a fault had been seen.

TABLE V
SELECTED ISSUES AND MITIGATIONS

Issue	Cause	Mitigation
Combine node crash	Shodan returned an error object	try/catch; default to empty object
Report node crash	Slack failure left fields undefined	Explicit guards; default upstream sources
Slack channel_not_found	Channel names rejected	Use Slack channel IDs
Jira field mismatch	Custom field absent in project	Remove field; use project key KAN, type Task
DNS ENOTFOUND	Local resolver failures	Flush DNS; use public resolvers
Gemini model 404	Model restricted for new accounts	Switch to gemini-1.5-flash

C. Lessons Learned

The development record yielded five practical lessons:

- (1) pin upstream data when debugging, because it isolates a failing node quickly;
- (2) set continue-on-fail on enrichment nodes, so that one API outage does not stop the response;
- (3) expect platform quirks: optional chaining and the Gemini credential test behaved inconsistently in n8n version 2.8.3, and an upgrade is recommended;
- (4) never paste real API keys into chat or documents, and revoke any key that has been exposed;
- (5) use mock endpoints first, so payload structure can be validated before touching production firewalls or EDR tools.

VI. EVIDENCE-SUPPORTED IMPLEMENTATION AND PROTOTYPE STATUS

Table VI lists the behaviors verified in the controlled local environment. These are team-reported functional observations, not independently reproduced measurements.

TABLE VI
VERIFIED MVP BEHAVIOR

Function	Observed result
Webhook ingestion	POST accepted; HTTP 200 returned
Normalization	Incident ID generated; alert type mapped
Enrichment	VT, AbuseIPDB and Shodan return data
Threat scoring	Composite score computed from all sources
AI assessment	Gemini returns structured JSON
Branching	IF node splits on autoContain flag
Containment	Requests reach three mock endpoints

Function	Observed result
Ticketing	Jira ticket created in KAN project
Audit log	Rows written to Sheets (two verified)
Final report	Consolidated JSON returned to caller

Pending items. Three items remain: switching the Gemini model to gemini-1.5-flash, replacing channel names with Slack channel IDs in the Slack node, and revoking and rotating the API keys exposed during development. The records do not establish real firewall, EDR or identity-provider integration, an approval gate, or load testing. The workflow should be described as a functionally verified proof of concept, not a production system.

Test procedure. Alerts were submitted with curl to the webhook using a JSON body containing source, alertType, sourceIP, destinationIP, affectedHost, affectedUser, timestamp and rawLog. A successful run produces a Slack message, a Jira ticket, a Sheets row, three mock-endpoint hits and a JSON report returned to the caller.

The next implementation stage should focus on reproducibility and safety rather than on adding features. The evaluated workflow export, prompts, node configurations, sample alerts, raw API responses, mock-endpoint payloads, Jira ticket references and Sheets rows should be preserved. The human approval gate should then be added, since it changes the risk profile of the containment branch more than any other single change. Only after that should real endpoints be introduced, first in a lab environment. These extensions are future work and should not be presented as features already implemented in the MVP.

VII. AVAILABLE EVALUATION AND PROSPECTIVE VALIDATION PROTOCOL

A. Available Evidence

Functional end-to-end runs were performed with test alerts covering. Each run was checked against the expected outputs listed above.

B. Prospective Protocol

A quantitative study should define each metric before testing and report it with its conditions and calculation method (Table VII). It should use realistic public IP indicators and varied alert types, keep a labeled set for severity agreement, and separate true, benign and malformed alerts as distinct cases.

TABLE VII
PROPOSED EVALUATION METRICS

Metric	Definition
End-to-end latency	Alert receipt to report returned, per stage and total
Workflow success rate	Runs completing all stages / total runs
Severity agreement	AI severity vs. analyst label on a labeled set
Containment errors	False-positive and false-negative containment rates
LLM consistency	Agreement across repeated runs of one alert
Resilience	Behavior under API outage and rate limits
Analyst time saved	Manual triage time vs. workflow time

Reproducibility requires preserving the workflow export, prompts, raw API responses, model version, decision thresholds and logs alongside the reported figures.

The present validation position is that the system has passed a preliminary functional check in a controlled local environment. It has not been evaluated for detection quality, for the correctness of the LLM verdicts or for behavior under load.

Any new metric should be defined before evaluation. The activities listed above are prospective validation and not completed evidence of operational reliability.

VIII. DISCUSSION AND DESIGN TRADE-OFFS

Automation versus control. Automatic containment shortens response time, but a wrong LLM judgment could block a legitimate IP or isolate a production host. A human approval step, implemented with a Wait node and a resume URL, is the recommended mitigation and reflects incident handling practice [1].

LLM flexibility versus determinism. LLM assessment handles varied alerts, but its output can drift or be malformed. Structured prompting and guarded parsing reduce this risk without removing it, so the threat score and the LLM verdict should be treated as separate signals.

Availability versus completeness. Continue-on-fail keeps the pipeline running through outages, at the cost of a lower-confidence composite score. The report should record which sources responded.

Prototype convenience versus security. Free-tier APIs and mock endpoints speed development but do not reflect production constraints such as rate limits, authentication of the webhook and secret management.

Transparency versus convenience. A single composite score and a single LLM verdict are easy for an analyst to read, but each hides how it was reached. Storing the individual source results, the score inputs and the model output with every incident makes decisions reviewable, at the cost of larger records. The current evidence supports a staged approach: verify the existing workflow, add the approval gate, and then extend testing to real indicators and labeled alerts.

IX. SAFETY, SECURITY, PRIVACY, ETHICS AND IP CONSIDERATIONS

Safety. Incorrect automated containment is the main failure mode. Mock endpoints should remain in place until an approval gate and a rollback procedure exist.

Security. Two API keys were exposed during development and must be revoked and rotated. Production use needs secret management, webhook authentication, network restrictions and least-privilege service accounts.

Privacy. Alerts may contain user identifiers and internal addresses, which the workflow sends to third-party services (enrichment APIs, the LLM provider, Slack, Jira and Google Sheets). Data minimization and provider terms should be reviewed before real data is processed.

Accountability. The audit log and per-incident report support review. The LLM's stated reasoning should be stored with each decision so that an analyst can challenge it.

IP and licensing. Terms for free-tier APIs, the n8n license for the intended use and any third-party data should be verified before external release.

Overall, the safety and ethical position of the project should remain proportional to its developmental status. The MVP demonstrates a controlled workflow, but it has not established the safety, privacy, security or regulatory properties required for a production SOC. The next stage should preserve the evidence, rotate credentials, add human oversight and review data-handling terms before processing real incident data.

X. LIMITATIONS

The evaluation was functional, small and run in a local environment. Containment endpoints are simulated. The sample alert uses a private address (192.168.1.100), which external reputation services cannot meaningfully assess, so enrichment on real public indicators remains to be demonstrated. No accuracy, latency or reliability figures are reported, and no comparison with existing SOAR tools was made. The model changed during development, so reported LLM behavior applies only to the model actually tested. The underlying logs and exports have not been independently re-verified.

A further limitation concerns the LLM stage. Severity and playbook recommendations come from a general-purpose model that was not tuned for security operations, and no analyst-labeled data was used to measure its agreement with human judgment. The composite threat score is likewise a heuristic whose weights have not been calibrated. The results should therefore be read as evidence that the pipeline runs, and not as evidence that its severity decisions are correct.

Finally, the manuscript does not treat the prospective validation activities as completed experiments. Until real indicators, larger and more varied alert sets, an approval gate and reproducible testing procedures are in place, the contribution should be understood as a controlled proof of concept and not as evidence of deployment-ready incident response automation.

XI. CONCLUSION AND FUTURE WORK

The project shows that a low-code orchestration platform can connect alert ingestion, threat intelligence, LLM-assisted assessment and automated response into an auditable pipeline. Results are limited to a controlled proof of concept. Future work will (i) add a human approval gate before containment, (ii) rotate credentials and adopt secret management, (iii) replace mock endpoints with real firewall, EDR and identity-provider APIs, (iv) upgrade the platform for stability, and (v) run a quantitative, reproducible evaluation on labeled alerts.

APPENDIX A: IMPLEMENTATION ISSUES LOG

TABLE A1
IMPLEMENTATION ISSUES AND FIXES

#	Issue	Fix applied
1	Four nodes accidentally deactivated	Re-activated each node with the pause/play toggle
2	DNS ENOTFOUND for AbuseIPDB and Gemini endpoints	Flushed DNS; switched to Google Public DNS
3	Combine node crash when Shodan returned an error	try/catch around each node reference; default to empty object
4	Gemini credential test reported failure	Ignored the test banner; saved the credential (workflow path is correct)
5	Gemini 2.5 models unavailable (404)	Switch to gemini-1.5-flash (in progress)
6	Slack channel_not_found	Replace channel names with channel IDs (in progress)
7	Report node crash on failed Slack response	Replace optional chaining with explicit guards; default sources
8	Jira custom field mismatch	Remove the field; project key KAN; issue type Task
9	PowerShell command failed in cmd.exe	Use curl with escaped quotes, or PowerShell
10	Sheets service account setup	Create account and key; enable APIs; share the sheet

APPENDIX B: TEST ALERT FIELDS

TABLE B1
FIELDS OF THE TEST ALERT

Field	Example value
source	splunk
alertType	brute_force
sourceIP / destinationIP	192.168.1.100 / 10.0.0.5
affectedHost	prod-server-01
affectedUser	admin@company.com
rawLog	Failed login attempt x50 in 60s

REFERENCES

- [1] P. Cichonski, T. Millar, T. Grance and K. Scarfone, "Computer security incident handling guide," NIST, Gaithersburg, MD, USA, Spec. Publ. 800-61 Rev. 2, Aug. 2012, doi: 10.6028/NIST.SP.800-61r2.
- [2] M. Vielberth, F. Böhm, I. Fichtinger and G. Pernul, "Security operations center: A systematic study and open challenges," *IEEE Access*, vol. 8, pp. 227756–227779, 2020.
- [3] B. E. Strom *et al.*, "MITRE ATT&CK: Design and philosophy," The MITRE Corporation, McLean, VA, USA, Tech. Rep., 2020.
- [4] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805, 2023.
- [5] n8n GmbH, "n8n documentation." [Online]. Available: <https://docs.n8n.io>
- [6] VirusTotal, "VirusTotal API documentation." [Online]. Available: <https://docs.virustotal.com>
- [7] AbuseIPDB, "AbuseIPDB API documentation." [Online]. Available: <https://docs.abuseipdb.com>
- [8] Shodan, "Shodan developer documentation." [Online]. Available: <https://developer.shodan.io>